

C1 FlashKDA: 官方 kernel 为什么停在 SM80 MMA

B300 复现、瓶颈量化与 SM100 tcgen05 挑战

Assignment 02 · Team C1

2026

从“新指令应该更快”的直觉，走到“形状与依赖决定性能”的实证结论

核心问题

对 CHUNK=16、小矩阵密集、chunk 间带状态依赖的 FlashKDA，在 B300 上把 SM80 `mma.sync` 直接换成 SM100 `tcgen05`，是否更快？

直觉

- ▶ B300 支持更新的 tensor-core 指令
- ▶ `tcgen05` 有 TMEM 和更强峰值能力
- ▶ “新指令”似乎应该替代“旧指令”

实测答案

- ▶ 正确性：PASS
- ▶ 原版：1.755 ms
- ▶ `tcgen05` 改版：17.552 ms
- ▶ 改版约慢 10.001 倍

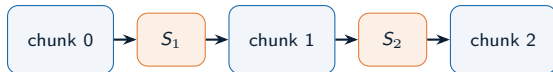
关键不是指令世代，而是算法粒度、物理 tile 与并行结构是否匹配。

KDA: 8192 个 token 为什么不能全部打散 ?

每个 head 的概念递推

$$S_t = d_t S_{t-1} + \Delta S_t, \quad o_t = q_t S_t$$

- ▶ 历史被压缩到 $D \times D$ 状态 S
- ▶ 同一 sequence、同一 head 的状态前后依赖
- ▶ FlashKDA 将序列分为长度 16 的 chunk

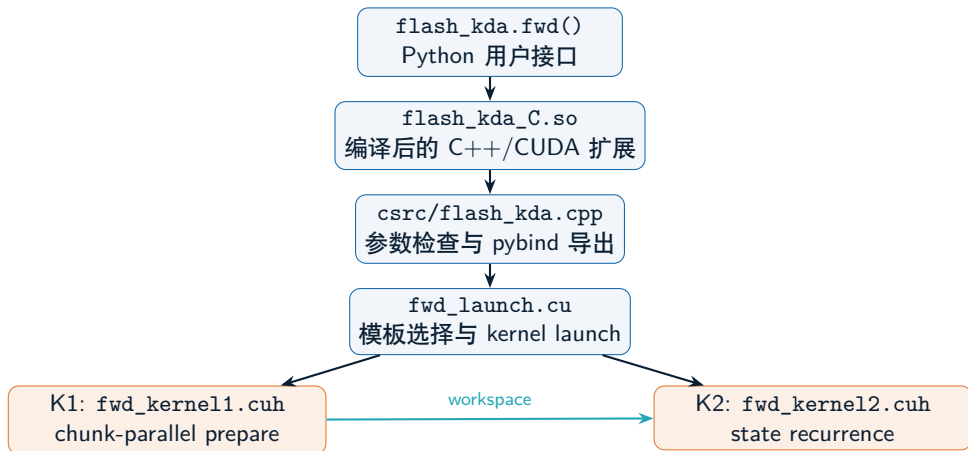


可并行: 不同 sequence、不同 head、chunk 内计算

不可直接并行: 同一 sequence/head 的相邻状态更新

fixed $T = 8192, H = 96$: 每条链有 $8192/16 = 512$ 个 chunk, K2 只有约 $1 \times 96 = 96$ 条独立链。

从 Python 到 GPU：项目结构与执行链



原版: `flash_kda` → `flash_kda_C`

改版: `flash_kda_tcgen05` →
`flash_kda_tcgen05_C`

两套包名允许在**同一进程**、**同一输入**、**同一张卡**上直接对拍。

复现：B300 上 FlashKDA 仍然很快

场景 ($T = 8192, H = 96, D = 128$)	FlashKDA	chunk_kda	对 chunk 加速	对 GDR 加速
fixed: 1×8192	1.0304 ms	2.5793 ms	2.503 ×	1.255×
varlen: 6 条不等长	0.8598 ms	2.6187 ms	3.046 ×	1.574×
varlen: 8×1024	0.6998 ms	2.5774 ms	3.683 ×	1.890×

复现可信度

B300 三个场景分别为 1.0304/0.8598/0.6998 ms, 与官方 GB200 的 1.0087/0.8597/0.7064 ms 接近。

状态链并行的直接证据

总 token 不变, 8×1024 比 1×8192 快:

$$1.0304/0.6998 = \mathbf{1.472} \times$$

证据: FlashKDA/BENCHMARK_GB200.md; B300 bench_fwd.py 日志

讨论 1: CHUNK=16 的三个约束

1. 数值范围

最坏 $g_t = -5$:

$$G_C = -5C$$

C	G_C	结果
16	-80	尚安全
32	-160	溢出/下溢
64	-320	更严重

2. Neumann 求逆

$$(I - L)^{-1} = \sum_{i=0}^{C-1} L^i$$

每 token 求逆 FLOP:

C	相对 16
16	1.00×
32	5.33×
64	26.67×

3. 指令形状

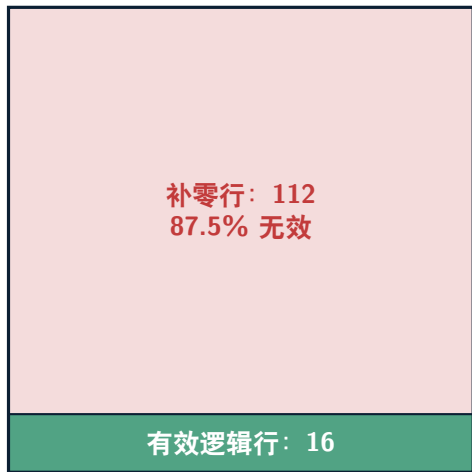
SM80:

m16n8k16

- ▶ M=16 正好匹配 chunk 行数
- ▶ 无需补到更大的物理 M
- ▶ 小 tile 固定成本低

最先破的是数值范围；即使增加 rescale，求逆和资源代价仍明显上涨。

讨论 2: 逻辑 M=16 遇到物理 M=128



tcgen05 物理 tile: $128 \times 128 \times 16$

- ▶ 单个 M=16 GEMM:

$$\text{issued/useful} = 128/16 = 8\times$$

- ▶ K2 每 chunk 动态发出 19 次 tcgen05:
 - kS^T : 8 次; qS^T : 8 次
 - $INV \cdot U$: 1 次; $Mqk \cdot U$: 1 次
 - state delta: 1 次 (唯一自然 M=128)

- ▶ 综合物理/有效行工作:

$$\frac{19 \cdot 128}{18 \cdot 16 + 128} = 5.846\times$$

- ▶ 仅约 17.1% 的物理行是有效计算

Microbench: 纸面浪费能否在机器上观察到？

形状	SM80 MMA	tcgen05	tcgen05/SM80	issued/useful
逻辑 M=16, 补到 M=128	0.006155 ms	0.012300 ms	0.500×	8.0×
自然 M=128	0.014542 ms	0.050765 ms	0.286×	1.0×

正确性

两组均: **PASS**, mismatches=0, max_abs=0。

为什么自然 M=128 仍可能慢？

问题很小且每次立刻同步读回, descriptor、TMEM、commit/wait 等固定成本没有被长流水摊薄。

microbench 不是最终结论, 而是决定是否值得接入真实 K2 的**中间证据**。

证据: experiments/instruction_only_mma.cu; B300 run_b300.sh 输出

讨论 4: 不是简单的 compute-bound / memory-bound

NCU 指标	K1 prepare	fixed K2 recurrence
Compute (SM) throughput	70.21%	20.6–21.9%
DRAM throughput	58.99%	18.6–19.6%
L2 throughput	72.32%	27–28%
Achieved occupancy	96.60%	9.37%
Eligible warps/scheduler	2.27	0.34–0.36
No eligible	28.55%	约 67.5%
Grid size	49,152	96
Dynamic shared/CTA	21.25 KiB	98.43 KiB

K1: 大量 chunk 独立, compute/cache/memory 都高, 属于高吞吐混合阶段。

fixed K2: 计算和 HBM 都没打满; 主因是**低 CTA 数、状态依赖、同步和资源限制**。

varlen 从 6 条增至 8 条后, K2 Compute 从约 28% 升至约 40%, DRAM 从约 25% 升至约 35%: **独立状态链越多, GPU 越容易被填满**。

证据: b300_fixed/varlen_h96_run1_details.txt

讨论 3: 状态有依赖, 并行度还能从哪里来?

候选方案	潜在收益	反例 / 风险
多 head 合一个 CTA	填满 $M=128$, 减少 padding	每 head 状态约 32 KiB; CTA 变少, 寄存器/共享内存压力增大
Persistent work queue +	改善 varlen 长短不均和尾部效应	fixed $N=1$ 时不能创造新状态链; 原版 K2 已在 CTA 内循环 chunk
2-CTA / cluster	可切 D 维, 流水 TMA/计算	每 chunk 的 cluster barrier 位于关键路径, 且资源翻倍
Affine prefix scan	理论上打破 chunk 串行深度	中间量、数值顺序和组合成本大, 属于算法重写

优先方向: 多序列/多 head 调度提高 useful tile; 若要真正突破单链依赖, 则需要算法级 scan/rescale。

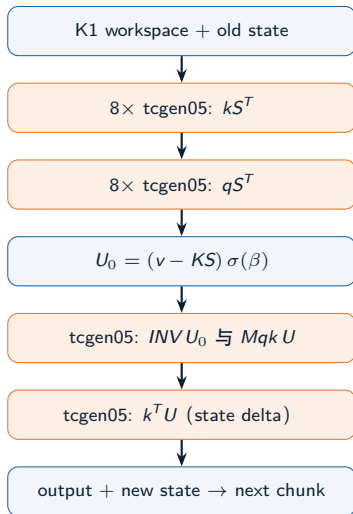
挑战实现：不是玩具 GEMM，而是真实 K2

保持不变

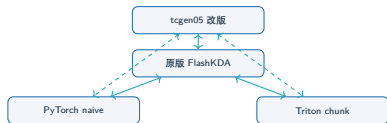
- ▶ KDA 数学语义与 CHUNK=16
- ▶ K1/K2 两阶段和 chunk 顺序
- ▶ gate、beta、状态落盘点
- ▶ Python API 输入输出

只替换 K2 GEMM 路径

- ▶ fwd_kernel12.cuh 接入 tcgen05
- ▶ tcgen05.cuh 封装 PTX/TMEM/mbarrier
- ▶ K1 仍保留原版 HMMA



正确性：为什么要四方对拍？



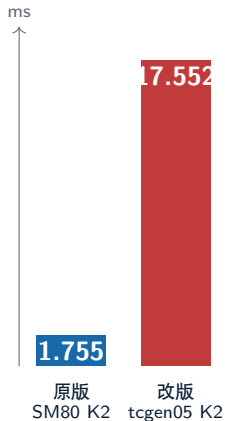
Case / 比较	output rel-RMSE	state rel-RMSE
fixed 64: 改版 vs 原版	0	0
fixed 64: 原版 vs naive	0.005537	0.004765
fixed 64: 原版 vs chunk	0.005410	0.004972
varlen [17,31,16]: 改版 vs 原版	0	0
varlen: 原版 vs naive	0.005886	0.004625
varlen: 原版 vs chunk	0.006235	0.004855

结论边界

fixed、varlen、非 16 倍数尾块均 **PASS**；tcgen05 未引入额外可见误差。但短序列通过**不等于** 8K/32K 极端 gate 的长期精度已经证明。

证据：FInal/results/correctness.log

真实 K2 结果：正确，但慢 10.001 倍



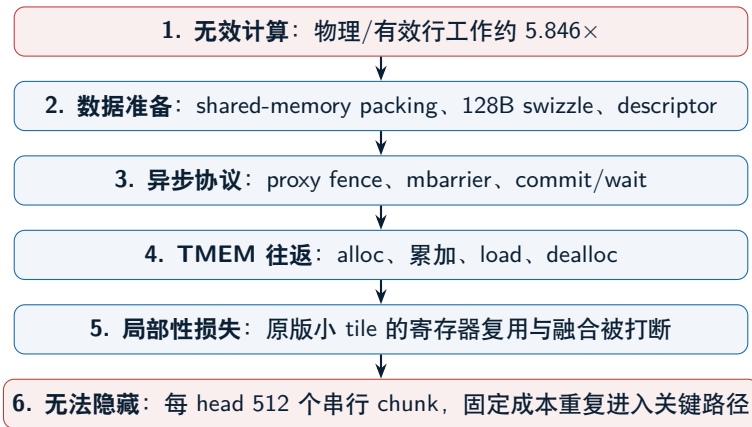
- ▶ 同进程、同输入、同卡 paired benchmark
- ▶ 改版/原版：10.001×
- ▶ 延迟增加：约 900.1%
- ▶ SASS：K2 recurrence 中出现 UTCHMMA
- ▶ K1 中仍有 HMMA：符合只改 K2 的范围

证据链

源码调用 tcgen05 → 编译为独立.so → K2 SASS 有 UTCHMMA → 正确性通过 → 完整 forward 被实际计时

证据：FInal/results/benchmark.log；sass/full.sass；key_instructions.txt

为什么实际慢 10 倍，而纸面浪费是 5.846 倍？



tcgen05 的峰值需要大而连续的流水；当前映射却是小而频繁同步的操作。

讨论 5: bf16 状态精度如何严谨验证？

必须分离的版本

1. Gold: FP64 state + FP64 update
2. A: FP32 state + FP32 update
3. B: BF16 state + FP32 accumulate (官方策略)
4. C: BF16 state + BF16 update (负面对照)

API 返回 fp32 不代表内部 recurrence 一直保存 fp32。

本次已有证据

短序列 fixed/varlen 相对 FP32 naive 的 output/state rel-RMSE 约 **0.46%–0.62%**；这是初步数据，不夸大全场景保证。

压力测试矩阵

- ▶ 长度: 16、256、1K、8K、32K
- ▶ gate: 近 0、近 -5、常数、交替极值
- ▶ beta/state scale: 正常与极端
- ▶ fixed/varlen/尾块, 至少 20 个 seeds
- ▶ output/state: rel-RMSE、max、P99/P99.9、cosine、NaN/Inf
- ▶ 画误差随 token/chunk 的增长曲线

讨论 6: v2 出不出 sm100a 专版 ?

继续研究的理由

- ▶ KDA 层数多, 小幅收益也有模型价值
- ▶ state-delta GEMM 天然匹配 $M=128$
- ▶ 多 head/序列打包可能提高 useful tile
- ▶ 可用 runtime dispatch 保留 fallback

当前不发布的理由

- ▶ 约 72.2% 逻辑 tensor FLOP 为 $M=16$
- ▶ K2 首要瓶颈是依赖/并行度, 不是峰值算力
- ▶ 原版已对 chunk_kda 加速 $2.50-3.68\times$
- ▶ 直接替换实测慢 $10.001\times$
- ▶ 架构专版增加维护与兼容成本

产品结论

不发布机械替换版。保留 SM80 默认路径; 只继续研究“大 CHUNK + 分段 rescale”或“多序列/多 head 打包 + persistent 调度”。真实端到端稳定收益达到 10%–15% 后再考虑 sm100a specialization。

结论：官方停在 SM80 是有量化依据的

1. **CHUNK=16** 同时满足数值范围、低成本求逆和 SM80 m16 tile。
2. B300 SASS 证明原版仍执行 `HMMMA.16816.F32.BF16`，且原版本身性能很强。
3. fixed K2 的关键瓶颈是**状态依赖与低并行度**，并非 tensor-core 峰值不足。
4. tcgen05 M=128 与逻辑 M=16 不匹配，综合物理/有效行工作为 **5.846×**。
5. 真实 K2 替换正确性通过，但完整 forward **慢 10.001×**。

最终回答

SM80 MMA 虽属于较早世代，但它的粒度和低固定开销更适合当前 FlashKDA。要发挥 SM100，必须先重构算法粒度或并行组织，而不是机械替换指令。

Q & A

正确的新指令，放在错误的形状上，也不会带来正确的性能。

代码: `FInal/FlashKDA-tcgen05/`

数据: `FInal/results/`